

Kotlin for Android

Build real apps with Jetpack Compose — a colorful pocket guide

🧩 Compose · 🎛️ State · 🎨 Material 3 · 🧭 Navigation · 🧠 ViewModel · 🌐 Retrofit · 🗄️ Room · 💉 Hilt · 🚀 Notes App

Book 2 · 13 chapters · code + phone-screen previews · exercises & cheat sheet

Contents

01 🤖 Android & Your First App 3

02 🔄 Activity Lifecycle & Logs 7

03 🧩 Compose Basics 10

04 📐 Layouts & Lists 14

05 🎛️ State in Compose 18

06 🎨 Material 3 UI 21

07 🧭 Navigation 25

08 🧠 ViewModel & StateFlow 29

09 ⚡ Coroutines on Android 33

10 🌐 Networking with Retrofit 37

11 🗄️ Room Database 42

12 💉 Dependency Injection with Hilt 45

13 🚀 Mini Project: Notes App 47

★ ⚡ Cheat Sheet 51

✓ ✅ Exercise Solutions 52

CHAPTER 1

Android & Your First App

Android Studio, project structure and Gradle

Android is the world's most used mobile OS, and **Kotlin is Google's recommended language** for it. Modern Android UI is built with **Jetpack Compose** — you describe the UI in Kotlin code, no XML layouts needed.

What you need

- **Android Studio** (free, from developer.android.com)
- An emulator or a real phone with *USB debugging* on
- The Kotlin fundamentals from the first book 😉

Create a project with **New Project → Empty Activity**. That template already uses Compose.

Project structure

Project tree

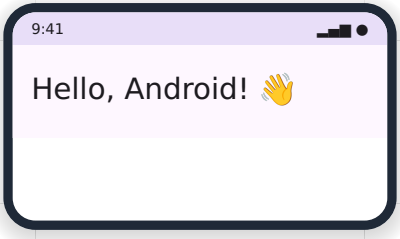
```
app/
 ├─ src/main/
 │   ├─ java/com/example/myapp/
 │   │   ├─ MainActivity.kt
 │   │   └─ ui/theme/          ← colors, fonts
 │   ├─ res/                   ← icons, strings
 │   └─ AndroidManifest.xml
 └─ build.gradle.kts           ← app settings
gradle/libs.versions.toml      ← library versions
```

Your first screen

MainActivity.kt

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState:
Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MyAppTheme {
                Greeting("Android")
            }
        }
    }
}

@Composable
fun Greeting(name: String) {
    Text(text = "Hello, $name! 👋")
}
```



📱 What you see

- `ComponentActivity` is one screen-hosting window of your app.
- `setContent { }` puts Compose UI on screen.
- `@Composable` marks a function that draws UI.

Gradle: adding libraries

app/build.gradle.kts

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
    alias(libs.plugins.kotlin.compose)
}

android {
    namespace = "com.example.myapp"
    compileSdk = 35
    defaultConfig {
        applicationId = "com.example.myapp"
        minSdk = 24
        targetSdk = 35
    }
    buildFeatures { compose = true }
}

dependencies {

implementation(platform(libs.androidx.compose.bom))
    implementation(libs.androidx.material3)
    implementation(libs.androidx.activity.compose)
}
```

💡 Tip

Library versions live in `libs.versions.toml`. Android Studio highlights outdated ones — press **Alt+Enter** to update. SDK numbers above are examples; new projects get the current ones automatically.

The manifest

AndroidManifest.xml

```
<manifest xmlns:android="http://schemas.android.com/
apk/res/android">
    <uses-permission
        android:name="android.permission.INTERNET" />
    <application
        android:label="My App"
        android:theme="@style/Theme.MyApp">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action
android:name="android.intent.action.MAIN" />
                <category
android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

📌 **Remember**

The manifest tells Android which screens exist, which one launches first, and which permissions the app needs.

CHAPTER 2

Activity Lifecycle & Logs



How Android starts, pauses and stops your app

Android controls your Activity's life. It calls special functions as the user opens, leaves and returns to your app.

Callback	When
onCreate	Activity created — set up UI here
onStart	Becoming visible
onResume	In front, user can interact
onPause	Partly hidden / leaving
onStop	No longer visible
onDestroy	Being destroyed

Logging with Logcat

MainActivity.kt

```
private const val TAG = "MainActivity"

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState:
Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d(TAG, "onCreate")
        setContent { Text("Watch Logcat!") }
    }
    override fun onStart()  { super.onStart();
Log.d(TAG, "onStart") }
    override fun onResume() { super.onResume();
Log.d(TAG, "onResume") }
    override fun onPause()  { super.onPause();
Log.d(TAG, "onPause") }
    override fun onStop()   { super.onStop();
Log.d(TAG, "onStop") }
}
```

▶ Logcat

```
D/MainActivity: onCreate
D/MainActivity: onStart
D/MainActivity: onResume
  (user presses Home)
D/MainActivity: onPause
D/MainActivity: onStop
```

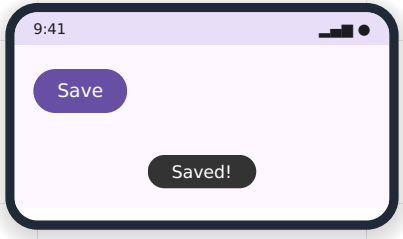
Function	Level
Log.v	Verbose
Log.d	Debug
Log.i	Info
Log.w	Warning
Log.e	Error

> ⚠️ **Watch out**
> Rotating the screen **destroys and recreates** the Activity by default. Data stored in plain variables is lost — that's why we use `rememberSaveable` and `ViewModel` (later chapters).

Toast — a quick popup

ToastButton.kt

```kotlin
@Composable
fun ToastButton() {
    val context = LocalContext.current
    Button(onClick = {
        Toast.makeText(context, "Saved!",
            Toast.LENGTH_SHORT).show()
    }) {
        Text("Save")
    }
}
```



📱 What you see

CHAPTER 3

Compose Basics

Composables, Text, Image and Modifiers



In Compose, UI is a tree of **composable functions**. When data changes, Compose re-runs (*recomposes*) only the parts that need updating.

Text styling

Title.kt

```
@Composable
fun Title() {
    Text(
        text = "Kotlin Rocks",
        fontSize = 28.sp,
        fontWeight = FontWeight.Bold,
        color = Color(0xFF7F52FF)
    )
}
```



📱 What you see

📌 Remember

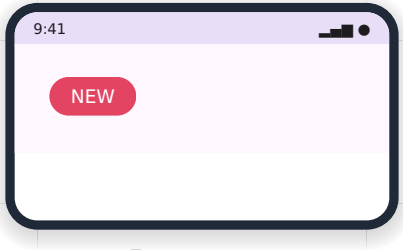
Use `.sp` for text sizes and `.dp` for everything else (padding, width, height). They scale on every screen.

Modifiers

A `Modifier` decorates a composable: size, padding, background, clicks... **Order matters!**

Badge.kt

```
@Composable
fun Badge() {
    Text(
        text = "NEW",
        color = Color.White,
        modifier = Modifier
            .padding(8.dp)                // outer
space
            .background(Color(0xFFE24462),
                RoundedCornerShape(12.dp))
            .padding(horizontal = 12.dp,  // inner
space
                     vertical = 4.dp)
            .clickable { /* do something */ }
    )
}
```



📱 What you see

Modifier	Does
fillMaxWidth()	Take the full width
size(48.dp)	Fixed width & height
padding(16.dp)	Space around
background(color)	Fill color
clip(CircleShape)	Cut into a shape
border(1.dp, color)	Outline
clickable { }	Make it tappable
weight(1f)	Share space in Row/Column

Images & icons

Avatar.kt

```
@Composable
fun Avatar() {
    Image(
        painter = painterResource(R.drawable.me),
        contentDescription = "My photo",
        contentScale = ContentScale.Crop,
        modifier = Modifier
            .size(64.dp)
            .clip(CircleShape)
    )
    Icon(Icons.Default.Favorite,
         contentDescription = "Like",
         tint = Color.Red)
}
```

> 💡 **Tip**
> Put images in `res/drawable`. `contentDescription` is read aloud by screen readers — always describe meaningful images.

@Preview

Previews.kt

```
@Preview(showBackground = true)
@Composable
fun TitlePreview() {
    MyAppTheme { Title() }
}
```

Previews render right inside Android Studio — no need to run the app!

CHAPTER 4

Layouts & Lists

Column, Row, Box and LazyColumn

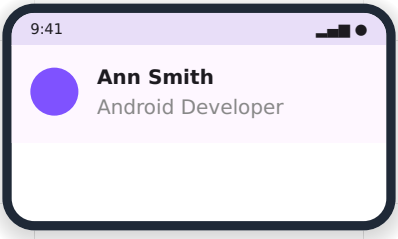


Layout	What it does
Column	Stacks children vertically ↓
Row	Places children horizontally →
Box	Layers children on top of each other
LazyColumn	Scrolling list, loads only visible items

Column & Row

ProfileCard.kt

```
@Composable
fun ProfileCard() {
    Row(
        verticalAlignment =
Alignment.CenterVertically,
        modifier = Modifier.padding(16.dp)
    ) {
        Box(
            Modifier.size(40.dp).clip(CircleShape)
                .background(Color(0xFF7F52FF))
        )
        Spacer(Modifier.width(12.dp))
        Column {
            Text("Ann Smith", fontWeight =
FontWeight.Bold)
            Text("Android Developer", color =
Color.Gray)
        }
    }
}
```



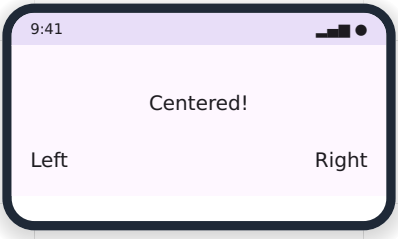
📱 What you see

Arrangement & alignment

Arrange.kt

```
Column(
    modifier = Modifier.fillMaxSize(),
    verticalArrangement = Arrangement.Center,
    horizontalAlignment = Alignment.CenterHorizontally
) {
    Text("Centered!")
}

Row(
    modifier = Modifier.fillMaxWidth(),
    horizontalArrangement = Arrangement.SpaceBetween
) {
    Text("Left"); Text("Right")
}
```



📱 What you see

Box — overlapping

Overlay.kt

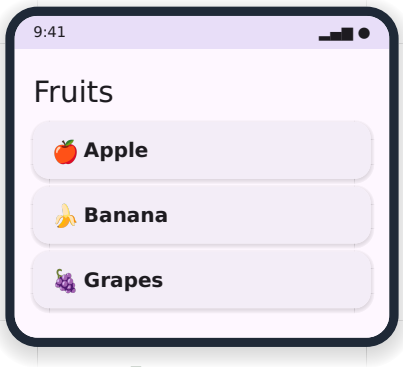
```
Box(Modifier.size(120.dp)) {
    Image(/* photo */, modifier =
Modifier.fillMaxSize())
    Text(
        "LIVE",
        modifier = Modifier.align(Alignment.TopEnd)
            .background(Color.Red).padding(4.dp),
        color = Color.White
    )
}
```

LazyColumn — scrolling lists

FruitList.kt

```
data class Fruit(val name: String, val emoji: String)

@Composable
fun FruitList(fruits: List<Fruit>) {
    LazyColumn(
        contentPadding = PaddingValues(16.dp),
        verticalArrangement =
Arrangement.spacedBy(8.dp)
    ) {
        item { Text("Fruits", fontSize = 22.sp) }
        items(fruits, key = { it.name }) { fruit ->
            Card(Modifier.fillMaxWidth()) {
                Text("${fruit.emoji}  ${fruit.name}",
                     Modifier.padding(16.dp))
            }
        }
    }
}
```



📱 What you see

💡 Tip

`LazyRow` works the same horizontally, and `LazyVerticalGrid` makes grids.

✍️ Try it yourself

Build a contact row: a colored circle with the first letter of the name, the name and phone number in a Column, and a 📞 icon pushed to the right using `Modifier.weight(1f)` .

CHAPTER 5

State in Compose

remember, mutableStateOf and state hoisting



State is any value that can change and should update the UI. When state changes, Compose redraws the affected composables automatically.

A counter

Counter.kt

```
@Composable
fun Counter() {
    var count by remember { mutableStateOf(0) }

    Column(horizontalAlignment =
Alignment.CenterHorizontally) {
        Text("$count", fontSize = 48.sp)
        Row {
            Button(onClick = { count-- }) {
Text("−") }
            Spacer(Modifier.width(8.dp))
            Button(onClick = { count++ }) {
Text("+") }
        }
    }
}
```



📱 What you see

- `mutableStateOf` creates observable state.

- `remember` keeps it across recompositions.
- `by` lets you use it like a normal variable.

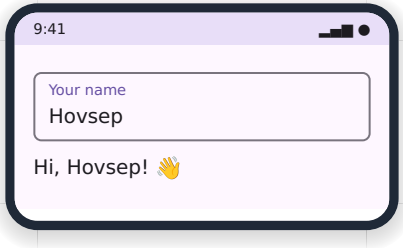
> ⚠️ **Watch out**
> `remember` forgets on screen rotation. Use `rememberSaveable` to survive configuration changes.

TextField input

NameInput.kt

```
@Composable
fun NameInput() {
    var name by rememberSaveable {
mutableStateOf("") }

    Column(Modifier.padding(16.dp)) {
        OutlinedTextField(
            value = name,
            onValueChange = { name = it },
            label = { Text("Your name") },
            singleLine = true
        )
        if (name.isNotBlank()) {
            Text("Hi, $name! 👋")
        }
    }
}
```



📱 What you see

State hoisting

Move state **up** to the caller, and pass down the value plus an event callback. This makes composables reusable and easy to test.

Hoisting.kt

```
// Stateless: just shows & reports
@Composable
fun LikeButton(liked: Boolean, onToggle: () -> Unit) {
    IconButton(onClick = onToggle) {
        Icon(
            if (liked) Icons.Filled.Favorite
            else Icons.Outlined.FavoriteBorder,
            contentDescription = "Like",
            tint = if (liked) Color.Red else
Color.Gray
        )
    }
}

// Stateful parent owns the state
@Composable
fun Post() {
    var liked by remember { mutableStateOf(false) }
    LikeButton(liked, onToggle = { liked = !liked })
}
```

📌 **Remember**

Golden rule: **state goes down, events go up.**

Lists of state

StateList.kt

```
val items = remember { mutableStateListOf("Milk") }
Button(onClick = { items.add("Eggs") }) {
    Text("Add")
}
```

CHAPTER 6

Material 3 UI

Scaffold, buttons, cards and theming



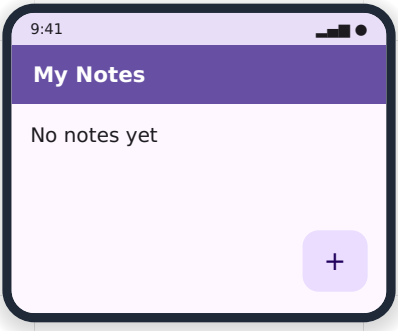
Material 3 is Google's design system. Compose gives you ready-made, beautiful components.

Scaffold — the screen skeleton

HomeScreen.kt

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun HomeScreen() {
    Scaffold(
        topBar = {
            TopAppBar(title = { Text("My Notes") })
        },
        floatingActionButton = {
            FloatingActionButton(onClick = { }) {
                Icon(Icons.Default.Add, "Add")
            }
        }
    ) { innerPadding ->
        Text("No notes yet",

Modifier.padding(innerPadding).padding(16.dp))
    }
}
```

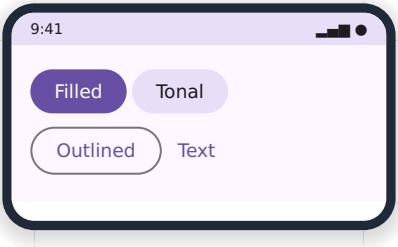


📱 What you see

> ⚠️ **Watch out**
> Always apply `innerPadding` — otherwise your content hides behind the top bar.

Buttons

```kotlin
Button(onClick = {}) { Text("Filled") }
FilledTonalButton(onClick = {}) { Text("Tonal") }
OutlinedButton(onClick = {}) { Text("Outlined") }
TextButton(onClick = {}) { Text("Text") }
```

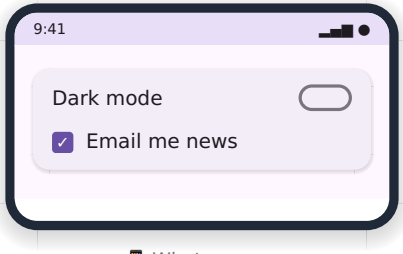


📱 What you see

Cards, switches & checkboxes

Settings.kt

```
@Composable
fun SettingsCard() {
    var dark by remember { mutableStateOf(false) }
    var news by remember { mutableStateOf(true) }
    Card(Modifier.fillMaxWidth().padding(16.dp)) {
        Column(Modifier.padding(16.dp)) {
            Row(verticalAlignment =
Alignment.CenterVertically) {
                Text("Dark mode", Modifier.weight(1f))
                Switch(checked = dark,
                       onCheckedChange = { dark =
it })
            }
            Row(verticalAlignment =
Alignment.CenterVertically) {
                Checkbox(checked = news,
                         onCheckedChange = { news =
it })
                Text("Email me news")
            }
        }
    }
}
```



📱 What you see

Theming

ui/theme/Theme.kt

```
private val LightColors = lightColorScheme(
    primary = Color(0xFF7F52FF),
    secondary = Color(0xFF3DDC84),
)

@Composable
fun MyAppTheme(content: @Composable () -> Unit) {
    MaterialTheme(
        colorScheme = if (isSystemInDarkTheme())
            darkColorScheme() else LightColors,
        typography = Typography(),
        content = content
    )
}

// use theme values anywhere:
Text("Hi", color = MaterialTheme.colorScheme.primary,
     style = MaterialTheme.typography.headlineMedium)
```

> 💡 **Tip**
> On Android 12+ you can use `dynamicLightColorScheme(context)` to match the user's wallpaper colors.

CHAPTER 7

Navigation



Moving between screens with Navigation Compose

Most apps have several screens. **Navigation Compose** handles moving between them and the back button. Add the `navigation-compose` library and the `kotlinx-serialization` plugin.

1. Define routes

Routes.kt

```
@Serializable
object Home

@Serializable
data class Details(val id: Int)
```

2. Build the NavHost

AppNav.kt

```
@Composable
fun AppNav() {
    val navController = rememberNavController()

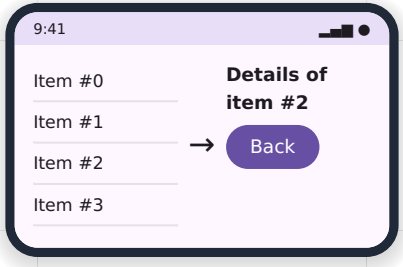
    NavHost(navController, startDestination = Home) {
        composable<Home> {
            HomeScreen(onItemClick = { id ->
                navController.navigate(Details(id))
            })
        }
        composable<Details> { entry ->
            val args = entry.toRoute<Details>()
            DetailsScreen(
                id = args.id,
                onBack = {
navController.popBackStack() }
            )
        }
    }
}
```

3. Screens

Screens.kt

```
@Composable
fun HomeScreen(onItemClick: (Int) -> Unit) {
    LazyColumn {
        items(5) { i ->
            ListItem(
                headlineContent = { Text("Item
#$i") },
                modifier = Modifier.clickable {
onItemClick(i) }
            )
        }
    }
}

@Composable
fun DetailsScreen(id: Int, onBack: () -> Unit) {
    Column(Modifier.padding(16.dp)) {
        Text("Details of item #$id", fontSize = 22.sp)
        Button(onClick = onBack) { Text("Back") }
    }
}
```



📱 What you see

> 📌 **Remember**
> Screens receive **callbacks** like `onItemClick` instead of the navController. This keeps them simple and previewable.

> 💡 **Tip**
> Bottom navigation: put a `NavigationBar` with `NavigationBarItem` s in the Scaffold's `bottomBar` , each navigating to a top-level route.

CHAPTER 8

ViewModel & StateFlow

Keep data alive and logic out of the UI



A **ViewModel** holds screen data and logic. It survives rotation, and keeps your composables clean. The UI **observes** state and **sends events**.

Architecture (UDF)

```
UI  ──events──▶  ViewModel  ──▶  Repository / Data
UI  ◀──state───  ViewModel
```

UI state as a data class

TodoViewModel.kt

```kotlin
data class TodoUiState(
    val input: String = "",
    val todos: List<String> = emptyList()
)

class TodoViewModel : ViewModel() {
    private val _state =
MutableStateFlow(TodoUiState())
    val state: StateFlow<TodoUiState> =
_state.asStateFlow()

    fun onInputChange(text: String) {
        _state.update { it.copy(input = text) }
    }

    fun addTodo() {
        _state.update {
            if (it.input.isBlank()) it
            else it.copy(
                todos = it.todos + it.input.trim(),
                input = ""
            )
        }
    }
}
```

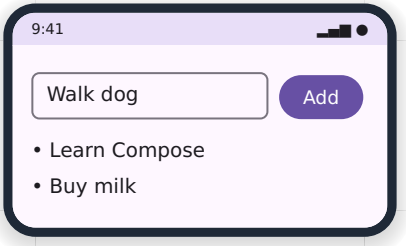
> 📌 **Remember**
> Expose a read-only `StateFlow`, keep the `MutableStateFlow` private. Only the ViewModel can change state.

Using it in Compose

TodoScreen.kt

```
@Composable
fun TodoScreen(vm: TodoViewModel = viewModel()) {
    val state by
vm.state.collectAsStateWithLifecycle()

    Column(Modifier.padding(16.dp)) {
        Row(verticalAlignment =
Alignment.CenterVertically) {
            OutlinedTextField(
                value = state.input,
                onValueChange = vm::onInputChange,
                modifier = Modifier.weight(1f)
            )
            Button(onClick = vm::addTodo) {
Text("Add") }
        }
        state.todos.forEach { Text("• $it") }
    }
}
```



📱 What you see

> 💡 **Tip**
>
> Needed libraries: `lifecycle-viewmodel-compose` (for `viewModel()`) and `lifecycle-runtime-compose` (for `collectAsStateWithLifecycle`).

One-time events

Events.kt

```
private val _messages = Channel<String>()
val messages = _messages.receiveAsFlow()

fun save() = viewModelScope.launch {
    _messages.send("Saved ✅")
}

// In the composable:
LaunchedEffect(Unit) {
    vm.messages.collect {
snackbarHost.showSnackbar(it) }
}
```

CHAPTER 9

Coroutines on Android

Background work without freezing the UI



The **main thread** draws the UI. Long work (network, database, files) on it freezes the app — Android may even show “App not responding”. Coroutines solve this.

Dispatcher	Use for
Dispatchers.Main	UI updates (default in viewModelScope)
Dispatchers.IO	Network, database, files
Dispatchers.Default	Heavy CPU work (sorting, parsing)

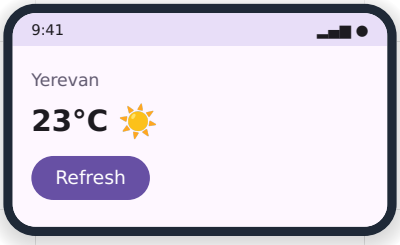
viewModelScope

Coroutines launched here are cancelled automatically when the ViewModel is cleared.

WeatherViewModel.kt

```kotlin
class WeatherViewModel : ViewModel() {
    var temp by mutableStateOf("--")
        private set

    fun refresh() {
        viewModelScope.launch {
            temp = "Loading..."
            val result = withContext(Dispatchers.IO) {
                delay(1500)          // pretend
network
                "23°C ☀️"
            }
            temp = result            // back on Main
        }
    }
}
```



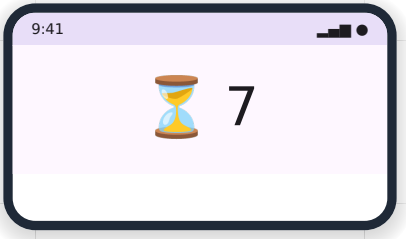
📱 What you see

LaunchedEffect — coroutines in composables

Countdown.kt

```
@Composable
fun Countdown() {
    var seconds by remember { mutableStateOf(10) }

    LaunchedEffect(Unit) {
        while (seconds > 0) {
            delay(1000)
            seconds--
        }
    }
    Text(if (seconds > 0) "⏳ $seconds" else "🎉
Done!",
         fontSize = 32.sp)
}
```



📱 What you see

📌 Remember

`LaunchedEffect(key)` starts when it enters the screen, and restarts if `key` changes. It's cancelled when it leaves.

rememberCoroutineScope — from clicks

ScrollTop.kt

```
val scope = rememberCoroutineScope()
val listState = rememberLazyListState()

Button(onClick = {
    scope.launch { listState.animateScrollToItem(0) }
}) { Text("Scroll to top") }
```

Flows

Flows.kt

```
fun ticker(): Flow<Int> = flow {
    var i = 0
    while (true) { emit(i++); delay(1000) }
}

val doubled = ticker()
    .map { it * 2 }
    .filter { it % 4 == 0 }
```

CHAPTER 10

Networking with Retrofit

Loading data from the internet



Retrofit turns a web API into a Kotlin interface. We'll use **kotlinx.serialization** to convert JSON into data classes.

> ⚠️ **Watch out**
> Add `<uses-permission android:name="android.permission.INTERNET"/>` to the manifest, or every request fails.

1. Model the JSON

Post.kt

```
// { "id": 1, "title": "Hello", "body": "..." }
@Serializable
data class Post(
    val id: Int,
    val title: String,
    val body: String
)
```

2. Describe the API

Network.kt

```
interface PostApi {
    @GET("posts")
    suspend fun getPosts(): List<Post>

    @GET("posts/{id}")
    suspend fun getPost(@Path("id") id: Int): Post
}

object Network {
    private val json = Json { ignoreUnknownKeys =
true }

    val api: PostApi = Retrofit.Builder()
        .baseUrl("https://
jsonplaceholder.typicode.com/")
        .addConverterFactory(
            json.asConverterFactory(
                "application/json".toMediaType()))
        .build()
        .create(PostApi::class.java)
}
```

3. Model screen states

PostsViewModel.kt

```
sealed interface PostsUiState {
    data object Loading : PostsUiState
    data class Success(val posts: List<Post>) :
PostsUiState
    data class Error(val message: String) :
PostsUiState
}

class PostsViewModel : ViewModel() {
    private val _state =

MutableStateFlow<PostsUiState>(PostsUiState.Loading)
    val state = _state.asStateFlow()

    init { load() }

    fun load() = viewModelScope.launch {
        _state.value = PostsUiState.Loading
        _state.value = try {

PostsUiState.Success(Network.api.getPosts())
        } catch (e: IOException) {
            PostsUiState.Error("No internet
connection")
        } catch (e: HttpException) {
            PostsUiState.Error("Server error $
{e.code()}")
        }
    }
}
```

4. Show every state

PostsScreen.kt

```
@Composable
fun PostsScreen(vm: PostsViewModel = viewModel()) {
    val state by
vm.state.collectAsStateWithLifecycle()

    when (val s = state) {
        PostsUiState.Loading ->
            Box(Modifier.fillMaxSize(),
Alignment.Center) {
                CircularProgressIndicator()
            }
        is PostsUiState.Error -> Column {
            Text("❌ ${s.message}")
            Button(onClick = { vm.load() }) {
Text("Retry") }
        }
        is PostsUiState.Success -> LazyColumn {
            items(s.posts) { post ->
                ListItem(
                    headlineContent = {
Text(post.title) },
                    supportingContent = {
                        Text(post.body, maxLines = 2)
                    }
                )
            }
        }
    }
}
```



📱 What you see

> 💡 **Tip**
> To load images from URLs, use the **Coil** library: `AsyncImage(model = url, contentDescription = null)` .

CHAPTER 11

Room Database

Saving data on the device



Room is a friendly layer over SQLite. Data stays after the app closes. It has three parts: **Entity**, **DAO** and **Database**.

Entity — a table

Note.kt

```
@Entity(tableName = "notes")
data class Note(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val title: String,
    val content: String,
    val createdAt: Long = System.currentTimeMillis()
)
```

DAO — the queries

NoteDao.kt

```
@Dao
interface NoteDao {
    @Query("SELECT * FROM notes ORDER BY createdAt
DESC")
    fun getAll(): Flow<List<Note>>

    @Insert
    suspend fun insert(note: Note)

    @Delete
    suspend fun delete(note: Note)

    @Query("SELECT * FROM notes WHERE title LIKE '%'
|| :q || '%'")
    fun search(q: String): Flow<List<Note>>
}
```

> 📌 **Remember**
> Returning `Flow` means the UI updates **automatically** whenever the table changes.

Database

AppDatabase.kt

```
@Database(entities = [Note::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun noteDao(): NoteDao

    companion object {
        @Volatile private var INSTANCE: AppDatabase?
= null

        fun get(context: Context): AppDatabase =
            INSTANCE ?: synchronized(this) {
                INSTANCE ?: Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "app.db"
                ).build().also { INSTANCE = it }
            }
    }
}
```

Using it in a ViewModel

NotesViewModel.kt

```
class NotesViewModel(private val dao: NoteDao) :
ViewModel() {

    val notes: StateFlow<List<Note>> = dao.getAll()
        .stateIn(viewModelScope,

SharingStarted.WhileSubscribed(5_000),
                emptyList())

    fun add(title: String, content: String) =
        viewModelScope.launch {
            dao.insert(Note(title = title, content =
content))
        }

    fun delete(note: Note) =
        viewModelScope.launch { dao.delete(note) }
}
```

Tip

Room needs the **KSP** plugin to generate code: `ksp(libs.androidx.room.compiler)` . For simple key/value settings, use **DataStore** instead.

Tip

ViewModels with constructor parameters need a factory — or use **Hilt** (next chapter) to provide them automatically.

CHAPTER 12

Dependency Injection with Hilt

Let Hilt create and share your objects



Instead of every class creating its own database and API, **Hilt** builds them once and *injects* them where needed. This makes code cleaner and easier to test.

1. Application class

MyApp.kt

```
@HiltAndroidApp
class MyApp : Application()

// AndroidManifest.xml:  <application
android:name=".MyApp" ...>
```

2. Tell Hilt how to build things

AppModule.kt

```
@Module
@InstallIn(SingletonComponent::class)
object AppModule {

    @Provides @Singleton
    fun provideDb(@ApplicationContext ctx: Context) =
        Room.databaseBuilder(ctx,
AppDatabase::class.java,
                             "app.db").build()

    @Provides
    fun provideNoteDao(db: AppDatabase) = db.noteDao()
}
```

3. Inject into classes

Injection.kt

```
class NoteRepository @Inject constructor(
    private val dao: NoteDao
) {
    val notes = dao.getAll()
    suspend fun add(note: Note) = dao.insert(note)
}

@HiltViewModel
class NotesViewModel @Inject constructor(
    private val repo: NoteRepository
) : ViewModel() {
    val notes = repo.notes.stateIn(
        viewModelScope,
SharingStarted.WhileSubscribed(5_000),
        emptyList())
}
```

4. Use it

MainActivity.kt

```
@AndroidEntryPoint
class MainActivity : ComponentActivity() { /* ... */ }

@Composable
fun NotesScreen(vm: NotesViewModel = hiltViewModel())
{
    val notes by
vm.notes.collectAsStateWithLifecycle()
    // ...
}
```

📌 **Remember**

Recommended layers: **UI (Compose)** → **ViewModel** → **Repository** → **Data sources** (Room, Retrofit).

CHAPTER 13

Mini Project: Notes App



Compose + ViewModel + StateFlow in one app

A complete, runnable notes app in one file. It uses an in-memory list so you can run it right away; swap the list for the Room DAO from Chapter 11 to save notes permanently.

MainActivity.kt

```
data class Note(val id: Int, val text: String,
                val done: Boolean = false)

class NotesViewModel : ViewModel() {
    private val _notes =
MutableStateFlow(listOf<Note>())
    val notes = _notes.asStateFlow()
    private var nextId = 1

    fun add(text: String) {
        if (text.isBlank()) return
        _notes.update { it + Note(nextId++,
text.trim()) }
    }
    fun toggle(id: Int) = _notes.update { list ->
        list.map { if (it.id == id) it.copy(done = !
it.done)
                   else it }
    }
    fun remove(id: Int) =
        _notes.update { l -> l.filterNot { it.id ==
id } }
}

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState:
Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
```

```
        setContent { MaterialTheme { NotesApp() } }
    }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun NotesApp(vm: NotesViewModel = viewModel()) {
    val notes by
vm.notes.collectAsStateWithLifecycle()
    var input by rememberSaveable {
mutableStateOf("") }

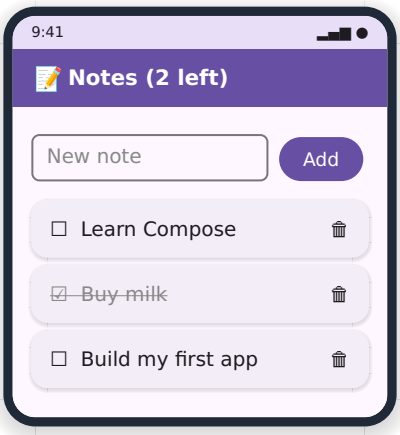
    Scaffold(topBar = {
        TopAppBar(title = {
            Text("📝 Notes (${notes.count { !
it.done }} left)")
        })
    }) { padding ->

Column(Modifier.padding(padding).padding(16.dp)) {
            Row(verticalAlignment =
Alignment.CenterVertically) {
                OutlinedTextField(
                    value = input,
                    onValueChange = { input = it },
                    placeholder = { Text("New
note") },
                    modifier = Modifier.weight(1f),
                    singleLine = true
                )
                Spacer(Modifier.width(8.dp))
                Button(onClick = { vm.add(input);
input = "" }) {
                    Text("Add")
                }
            }
            Spacer(Modifier.height(12.dp))
            LazyColumn(verticalArrangement =

Arrangement.spacedBy(8.dp)) {
                items(notes, key = { it.id }) { note -
>
                    NoteRow(note,
                        onToggle = {
vm.toggle(note.id) },
                        onDelete = {
```

```
vm.remove(note.id) })
                }
            }
        }
    }
}

@Composable
fun NoteRow(note: Note, onToggle: () -> Unit,
            onDelete: () -> Unit) {
    Card(Modifier.fillMaxWidth()) {
        Row(Modifier.padding(8.dp),
            verticalAlignment =
Alignment.CenterVertically) {
            Checkbox(note.done, onCheckedChange = {
onToggle() })
            Text(
                note.text,
                Modifier.weight(1f),
                textDecoration = if (note.done)
                    TextDecoration.LineThrough else
null
            )
            IconButton(onClick = onDelete) {
                Icon(Icons.Default.Delete, "Delete")
            }
        }
    }
}
```



📱 What you see

✍️ Try it yourself

1) Add a filter row with “All / Active / Done” chips (`FilterChip`).
2) Show a Snackbar with **Undo** after deleting a note.
3) Replace the in-memory list with Room.

⚡ Android Cheat Sheet

Compose & Jetpack at a glance

Task	Code
Show UI	`setContent { App() }`
Composable	`@Composable fun X() { }`
State	`var x by remember { mutableStateOf(0) }`
Survive rotation	`rememberSaveable { }`
Vertical / horizontal	`Column { }` / `Row { }`
Scroll list	`LazyColumn { items(list) { } }`
Sizes	`16.dp`, text `18.sp`
Input	`OutlinedTextField(value, onValueChange)`
Screen skeleton	`Scaffold(topBar = { })`
Theme color	`MaterialTheme.colorScheme.primary`
Navigate	`navController.navigate(Details(1))`
Go back	`navController.popBackStack()`
ViewModel	`vm: MyVm = viewModel()`
Observe flow	`flow.collectAsStateWithLifecycle()`
Background work	`viewModelScope.launch { }`
Switch thread	`withContext(Dispatchers.IO) { }`
Effect	`LaunchedEffect(key) { }`
Context	`LocalContext.current`
Log	`Log.d("TAG", "msg")`
Room	`@Entity @Dao @Database`
Hilt	`@HiltViewModel @Inject constructor`

ANSWERS

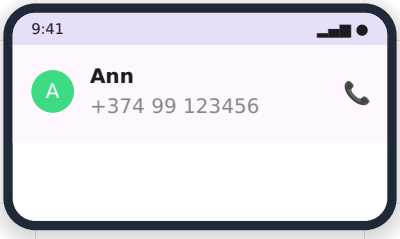
Exercise Solutions

Compare with your own code

Contact row (Ch. 4)

ContactRow.kt

```
@Composable
fun ContactRow(name: String, phone: String) {
    Row(
        Modifier.fillMaxWidth().padding(12.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Box(
            Modifier.size(40.dp).clip(CircleShape)
                .background(Color(0xFF3DDC84)),
            contentAlignment = Alignment.Center
        ) {
            Text(name.first().uppercase(),
                 color = Color.White)
        }
        Spacer(Modifier.width(12.dp))
        Column(Modifier.weight(1f)) {
            Text(name, fontWeight = FontWeight.Bold)
            Text(phone, color = Color.Gray)
        }
        Text("📞", fontSize = 22.sp)
    }
}
```



📱 What you see

Filter chips (Mini Project)

Filters.kt

```
enum class Filter { ALL, ACTIVE, DONE }

var filter by rememberSaveable {
mutableStateOf(Filter.ALL) }

Row(horizontalArrangement =
Arrangement.spacedBy(8.dp)) {
    Filter.entries.forEach { f ->
        FilterChip(
            selected = filter == f,
            onClick = { filter = f },
            label = { Text(f.name.lowercase()
                .replaceFirstChar {
it.uppercase() }) }
        )
    }
}

val shown = when (filter) {
    Filter.ALL -> notes
    Filter.ACTIVE -> notes.filter { !it.done }
    Filter.DONE -> notes.filter { it.done }
}
```

Undo with Snackbar (Mini Project)

Undo.kt

```
val snackbar = remember { SnackbarHostState() }
val scope = rememberCoroutineScope()

// Scaffold(snackbarHost =
{ SnackbarHost(snackbar) }, ...)

onDelete = {
    vm.remove(note.id)
    scope.launch {
        val r = snackbar.showSnackbar(
            "Note deleted", actionLabel = "Undo")
        if (r == SnackbarResult.ActionPerformed)
            vm.restore(note)
    }
}

// In NotesViewModel:
fun restore(note: Note) =
    _notes.update { (it + note).sortedBy { n ->
n.id } }
```

🎉 You can now build real Android apps with Kotlin!
Next steps: publish an app on Google Play, learn testing (JUnit & Compose UI tests), WorkManager, and explore the official **developer.android.com** codelabs.